INFO-H420 Process Modeling, Data Pipelines, and Responsible AI

Assignment 3: Workflows with Airflow (10 points)

The goal is to build a small local pipeline that turns research PDFs into a searchable study library. The assignment focuses on learning **Apache Airflow** and calling a **locally served LLM**.

What to build

1. Run Airflow locally using `airflow standalone`.
2. Keep PDFs in a Google Drive-synced `papers/` folder.
3. Create one scheduled Airflow DAG that looks for new PDFs.
4. Convert each PDF to Markdown.
5. Ask a locally served LLM to generate a title, summary, keywords, and category as JSON.
6. Parse the JSON and save the metadata to a local SQLite database.
7. Regenerate a static HTML page listing the papers, with simple search or category filtering.
8. Send a Discord webhook notification when a new paper has been processed.

Google Drive syncs the PDFs and generated HTML. Keep the SQLite database on the local machine, outside the synced folder.

Suggested folder layout

```
Google Drive/
└── ResearchLibrary/
    ├── papers/
    │   └── paper.pdf
    └── output/
        └── index.html

Local project/
├── dags/
│   └── paper_pipeline.py
└── data/
    └── papers.db
```

Suggested tools

- **Airflow:** Standalone mode
- **PDF to Markdown:** PyMuPDF4LLM
- **Local LLM serving:** Ollama OpenAI compatibility or LM Studio OpenAI compatibility
- **LLM client:** Python's `openai` package can call an OpenAI-compatible local endpoint.
- **Database:** Python's built-in `sqlite3`
- **Discord notifications:** A Discord webhook

LLM output

Ask the model to return JSON with this structure:

```
{
  "title": "Paper title",
  "summary": "A short summary of the paper.",
  "keywords": ["keyword one", "keyword two"],
  "category": "Data Engineering"
}
```

Use JSON mode or structured output if the chosen model and server support it. Parse the response with Python's `json.loads()`. If parsing fails, retry once. Check that the required fields exist and have the expected basic types; extensive validation is not required.

Database

Use `sqlite3` to store one record per paper. A simple table can contain:

- filename (unique)
- title
- summary
- keywords (stored as a JSON string)
- category

Assume filenames are unique.

Assessment

Criterion	Weight
Airflow DAG and task dependencies	3 points
PDF conversion and local LLM integration	3 points
SQLite storage, static HTML output, Discord notification	2 points
Project report and documentation	2 points

Instructions

The assignment contributes 10% to the overall grade (i.e., 2/20).

This assignment is to be made in **groups of three** persons. You are asked to form the groups via “Groups for Assignment 3” on the Université Virtuelle (UV). If you cannot find a partner, please post a request in the “Discussion Forum” on UV. You should not form the same group as in the previous assignments.

You are asked to submit a short **report** presenting your proposed solution. The report should be short, discussing the approach you take, and showcasing that the workflow executes correctly. It should also contain link to a Github repository that should contain the assignment code, appropriate README, and run instructions.

The report has to be uploaded as a pdf file to “Assignment 3” on UV by **November 9, 2026**.